DigitalTwin.ai

Accenture Innovation Challenge 2026 — Problem Track 4

Team HipHipHooray · IIT Kharagpur

A live digital twin of a mixed-model vehicle assembly line that shows a floor supervisor **which station is holding the line back right now**, **which tool is drifting toward scrap**, and — the part most twins skip — **what to do about it and what ignoring it costs**.

It is built for the line the brief actually describes: a patchwork of legacy and modern equipment where **some stations have no sensors at all**.

Run it

```
pip install -r requirements.txt
python web/server.py --speed 120 --shifts 0
```

Then open **http://127.0.0.1:8080**.

That is the whole setup. A sample shift ships in `sample_data/`, so this works from a clean clone with **no arguments and no dataset download**. Only the *observed* logs are shipped — no ground-truth files, because a detector that can read the answer key is not a detector.

An 8-hour shift replays in about 4 minutes at `--speed 120`. `--shifts 0` runs shift after shift indefinitely, carrying the alert ledger across each boundary.

The problem, precisely

A plant already knows its weekly average throughput. What it does not know is **which station is the constraint at 14:20 today** — because the constraint moves roughly 20 times per shift, and an average describes no single moment of it. Meanwhile a defect introduced at station 5 may not surface until end-of-line inspection 40 stations later, by which time a hundred vehicles carry it.

Both problems are hard for the same reason: **the data is uneven.** Some stations stream torque, angle and motor current; others are a human with a clipboard.

Solution approach

The twin is built on four ideas, in order of how much they matter.

1 · Causality is enforced structurally, not promised. The replay driver truncates the data at the current timestamp, so the detector *cannot* read `t > now` even by accident. This is a property of the object it is handed, not a rule the code is trusted to follow — the class of bug that quietly inflates most published results is made unrepresentable.

2 · Dark stations are inferred, not skipped. A station with zero instrumentation is bracketed between the buffers either side of it. If work is piling up upstream and draining downstream, that station is slowing — no

sensor required. Measured across 120 runs, **15.5% of all forming warnings name a station with no sensors at all** [15.0–15.9], n=29,060.

3 · An alert must be able to justify itself. Every alert carries five fields — candidate and margin, evidence, persistence, recommended action, and cost of ignoring it. **An alert that cannot fill all five is suppressed, not downgraded.** The count of suppressed alerts is shown on screen, because silence should be legible.

4 · Confidence is calibrated, not asserted. Fitted on 30 runs and reported on 20 disjoint held-out runs, expected calibration error falls **0.479** → **0.025**. Uncalibrated, the detector claimed 0.997 confidence at a true hit rate near 0.11.

Architecture

```
sample_data/ or dataset/            observed logs only
         │
         ▼
record.py — Recorder.view_at(t)     truncates the run at t — causality
         │
         ▼
loop.py — TwinLoop.tick(t)          one JSON frame per tick:
         │                          ingest → detect → rank → prescribe → emit
         ├── detect.py              which station is the constraint
         ├── forming.py             buffer countdowns — what is forming next
         ├── dark.py                inference where there are no sensors
         ├── genealogy.py           tool faults, onset, containment
         └── rollup.py              shift / week / leadership aggregation
         │
         ├──────────────► store.py    SQLite, 7 tables, WAL
         ├──────────────► sink.py     append-only JSONL (crash-proof raw stream)
         │
         ▼
web/server.py                       FastAPI; a server-side driver task owns
         │                          the loop, viewers subscribe to a broadcast
         ▼
web/index.html                      one page, three stakeholder views
```

The driver is server-side on purpose. The plant advances whether or not a browser is attached; opening or closing a tab changes nothing about what is recorded. (It did not always work this way — see `PROGRESS.md`.)

Module map

Module	Responsibility
`record.py`	Replay driver. `view_at(t)` is where causality is enforced
`loop.py`	The tick: one frame, one verdict, one prescription, the alert contract
`detect.py`	Constraint detection and station ranking
`forming.py`	Buffer-slope countdowns — what becomes the constraint next
`dark.py`	Inference at uninstrumented stations
`genealogy.py`	Two-sided CUSUM, onset dating, containment, stop-or-continue

Module	Responsibility
`rollup.py`	Supervisor → manager → leadership aggregation and reconciliation
`store.py`	SQLite persistence, 7 tables, WAL so reads never block the writer
`sink.py`	JSONL append-only stream
`plant.py`, `line.py`, `layouts.py`, `tools.py`	The simulator that generates the data

HTTP surface

Endpoint	Purpose
`GET /`	The dashboard
`GET /meta`	Line topology and which stations are dark
`GET /stream`	Server-sent events — one frame per tick
`POST /alert/{index}/{outcome}`	Supervisor confirms or overrides an alert
`GET /genealogy`	Tool assessments, onset, containment
`GET /rollup`	Manager and leadership aggregates
`GET /alerts`	Alert ledger and calibration state
`GET /recording`	Persistence status

Key features

Prescriptive, not just predictive. The twin does not stop at "S04 is the constraint." It names the action, and states what not acting costs in vehicles. The action vocabulary is keyed on fault signature — slowing, breakdown, starved, blocked — and the cost is always computed, never retrieved from a table.

Same symptom, opposite correct answers. Two tools both show torque moving. One is genuine wear (*service it*); the other is a lying transducer whose rejections are false (*recalibrate only — servicing it scraps good parts and fixes nothing*). They are separated by asking whether a mechanically coupled channel moved with the torque. Across 1,246 alarmed tools, **70.5% of assessments are actionable** rather than "unclear" [67.9–72.9].

Containment partitioned by location. When onset is dated backwards off the CUSUM accumulator, the affected vehicles are split into those still on the line and those already shipped — because one is a rework instruction and the other is a customer event.

Three views, one record stream. Supervisor (real-time), manager (weekly distributions, deliberately *not* averages), leadership (investment case). They are proved to be one twin by a **reconciliation test**: each level totals independently and must match exactly. It passes at 6,730 constraint-minutes and 4,431 vehicles.

A trust ledger. Every alert is logged with the supervisor's confirm or override, so precision can be shown *over time* rather than claimed once. Alert volume is **48.6 per shift** (range 20–102) against the ISA-18.2 budget of 150.

ISA-101 HMI, verified. Grey base, colour reserved for deviation. All 16 colour pairings meet WCAG AA contrast; the minimum type size is 12px and keyboard focus is visible.

Read-only by design. The twin advises and never writes to line control — a read-only boundary in ISA-95 terms, stated on every screen. Sensing additions mount externally, so no PLC program changes and no re-validation.

Deploying this on a real line

The integration surface is five streams a plant already produces. Every module is written against that contract rather than against our simulator, which is why the same code runs on both.

What the twin reads	Where a plant already has it
Unit scans — VIN, station, timestamp	Barcode/RFID readers at station boundaries, already mandatory for traceability
Station state transitions	PLC state tags, already historised for OEE reporting
Buffer levels	Conveyor counters and occupancy sensors
Tool readings — torque, angle, current	Nutrunner controllers over Open Protocol / OPC-UA
Andon, rework, calendar	The andon system and the MES

Nothing on that list requires new hardware. Day-one deployment buys no sensors — the retrofit schedule is what the twin earns later, once it has measured which uninstrumented station is actually costing money.

The boundary is enforced, not promised

`tests/test_readonly_boundary.py` walks the syntax tree of every module on every test run and asserts that:

- only `store.py` and `sink.py` write anything at all, and only to our own store
- **no module may import a network client** — not `socket`, not `opcua`, not `pymodbus` — so the twin cannot reach a PLC even by mistake
- every SQL write names a table the twin itself owns
- every emitted frame is labelled advisory

We verified the test can fail by planting a PLC write and a socket import and confirming it was caught.

What stands between this and a live line

One adapter. The prototype reads these streams from files; there is no historian or MES connector. That is the honest gap, and it is deliberately the only one — connecting a real plant means writing one module that subscribes to that plant's historian and emits these tables. Nothing downstream changes.

Shadow mode already works today. Point the replay driver at a plant's exported logs and the entire twin runs, with no live connection of any kind. That is Phase 0 of the roadmap, and it is the phase we can already execute.

What it does not do

Stated here rather than discovered by a reader:

- **Operator variation is excluded deliberately, on ethical grounds.** The brief names it as a root cause. We measure the station, never the person.
- **Equipment vintage is not modelled** — there is no data axis for it.

- **A CONWIP lead-time figure was deleted from the business case** because no file in `results/` produced it.
- **We do not beat every baseline.** McNemar on identical blocks: we significantly beat a utilisation baseline (p=0.0025) and are **statistically tied** with an active-period method (p=0.45). We claim the first only.
- **One mechanism was measured and killed** — an overtake-risk predictor that was correct 5.9% of the time against 70–100% stated confidence.
- **There is no historian/MES adapter.** The input streams are read from files. See *Deploying this on a real line* above.

Validation

```
pip install -r requirements-dev.txt
pytest tests/ -q                    # skips need the regenerable dataset/
```

Transfer across four line topologies, including a parallel-server pair that breaks the pure-series assumption:

Line	n blocks	Top-1	Regret (cars)
L1 — 20 stations, 1 merge *(reference)*	191	44.5%	1.208
L2 — 30 stations, 2 merges	94	10.6%	1.218
L3 — 20 stations + parallel pair	94	13.8%	1.431

Top-1 accuracy collapses 34 points; regret moves 0.22. The operationally meaningful metric transfers and the vanity metric does not — which is why regret is the headline number throughout this project. Top-1 is additionally noise-dominated: label noise alone is 0.79 cars, making the argmax a coin flip in roughly half of all blocks.

Repository layout

Path	Contents
`src/twin/`	The twin and the simulator
`web/`	FastAPI server and the dashboard
`sample_data/`	One shift, observed logs only — the demo runs on this
`scripts/`	Dataset builders, evaluations, calibration fitting, analysis
`tests/`	Test suite
`results/`	Measured outputs every claim above is sourced from
`docs/dataset/`	Dataset documentation
`8_Proposal/`	Detailed Business Proposal — PDF and PPT, same content, both formats
`DEMO_RUNBOOK.md`	Measured beat timeline for the demonstration video
`PROGRESS.md`	Full engineering log — every decision, bug and correction
`1_` – `7_`	Round 1 submission and design documents

The generated datasets (~765 MB) are not committed; the seeded builders in `scripts/` regenerate them exactly.

Reference parameters

The brief assumes 30–50 stations across body, paint and final assembly, with a majority well-instrumented and a meaningful minority on manual checks. On the 40-station segmented layout, body and final assembly — the 30 stations where per-unit measurement is meaningful — sit at **75.3% instrumented, 24.7% manual**. Paint sits at 90% uninstrumented *by design*: booth-level sampling is how paint shops work, a property of the process rather than a coverage gap.

Manual checks are not treated as a sensor. They enter as attested data carrying their entry latency, and the twin tests whether they are honest: across 31,329 entries the checklist passes 96.51% of vehicles, yet **835 of those went on to fail end-of-line — a 2.76% escape rate.** A checklist reading near-100% against a non-zero failure rate is measuring compliance with the checklist, not quality.

License